

22IS68L-DATA MINING LABORATORY

Instruction : Consider German Credit Dataset for solving lab programs

- 1. List all the categorical (or nominal) attributes and the real-valued attributes separately. Attributes:- 1. checking_status 2. credit history 3. purpose 4. savings_status 5. employment 6. personal status 7. debtors 8. property 9. installment plans 10. housing 11. job 12. telephone 13. foreign worker 14. duration 15. credit amount 16. credit amount 17. residence 18. age 19. existing credits 20. num_dependents**

Sl. No.	Attribute Name	Type
1	checking_status	Categorical (Nominal)
2	credit_history	Categorical (Nominal)
3	purpose	Categorical (Nominal)
4	savings_status	Categorical (Nominal)
5	employment	Categorical (Nominal)
6	personal_status	Categorical (Nominal)
7	debtors	Categorical (Nominal)
8	property	Categorical (Nominal)
9	installment_plans	Categorical (Nominal)
10	housing	Categorical (Nominal)
11	job	Categorical (Nominal)
12	telephone	Categorical (Nominal)
13	foreign_worker	Categorical (Nominal)
14	duration	Real-Valued / Numeric

Sl. No.	Attribute Name	Type
15	credit_amount	Real-Valued / Numeric
16	installment_commitment	Real-Valued / Numeric
17	residence	Real-Valued / Numeric
18	age	Real-Valued / Numeric
19	existing_credits	Real-Valued / Numeric
20	num_dependents	Real-Valued / Numeric

2. What attributes do you think might be crucial in making the credit assessment ?
Come up with some simple rules in plain English using your selected attributes.

Crucial Attributes for Credit Assessment

The following attributes are usually very important in deciding whether a customer is a good credit risk:

Attribute	Why It Is Important
credit_history	Shows whether the customer repaid previous loans properly
checking_status	Indicates current financial condition and bank balance
savings_status	Reflects financial security and backup savings
employment	Stable employment suggests regular income
duration	Longer loan duration may increase repayment risk
credit_amount	Large loan amounts may be harder to repay
age	Older applicants may have more financial stability
existing_credits	Too many existing loans increase risk
job	Stable and skilled jobs generally indicate reliable income
foreign_worker	Sometimes used in risk evaluation policies

Attribute	Why It Is Important
housing	Owning a house may indicate financial stability

Simple Credit Assessment Rules

Rule 1

If the applicant has a **good credit history** and a **stable job**, then approve the loan.

Rule 2

If the applicant requests a **very high credit amount** and has **low savings**, then the application is risky.

Rule 3

If the applicant has **many existing credits** and a **long loan duration**, then reject the loan.

Rule 4

If the applicant has **strong savings** and is employed for a **long period**, then approve the loan.

Rule 5

If the applicant has a **poor credit history**, then reject the application regardless of other factors.

Rule 6

If the applicant is **young**, has **unstable employment**, and requests a **large loan amount**, then mark as high risk.

Rule 7

If the applicant owns a **house/property** and has a **moderate credit amount**, then the customer is likely reliable.

Rule 8

If checking account status is **low or negative** and there are **multiple existing loans**, then reject the credit request.

Rule 9

If the applicant has **regular employment**, **small loan duration**, and **small credit amount**, then approve the loan.

Rule 10

If savings are **high** but credit history is **bad**, then perform further verification before approval.

3. Train a Decision Tree using the complete dataset as the training data. Report the model obtained after training.

Program:

```
pip install pandas scikit-learn
# Import required libraries
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text

# Load dataset
# Replace the file name with your dataset file
data = pd.read_csv("german_credit.csv")

# Display first few rows
print("Dataset Sample:")
print(data.head())

# Convert categorical attributes into numeric values
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Separate features and target class
# Assume last column is the target attribute
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Train Decision Tree using complete dataset
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X, y)
```

```
# Print trained decision tree
tree_rules = export_text(model, feature_names=list(X.columns))

print("\nDecision Tree Model:\n")
print(tree_rules)
```

Expected Output :

	checking_status	duration	credit_amount	...
0	1	6	1169	
1	2	48	5951	
2	4	12	2096	
...				

Decision Tree Model:

```
|--- credit_history <= 1.50
| |--- savings_status <= 2.50
| | |--- class: 1
| |--- savings_status > 2.50
| | |--- class: 0
|
|--- credit_history > 1.50
| |--- duration <= 24.50
| | |--- class: 1
| |--- duration > 24.50
| | |--- class: 0
```

4. Suppose you use your above model trained on the complete dataset, and classify credit good/bad for each of the examples in the dataset. What % of examples can you classify correctly? Why do you think you cannot get 100 % training accuracy?

Expected Answer : **Training Accuracy Using Complete Dataset**

The Decision Tree model was trained using the complete German Credit Dataset and the same dataset was also used for testing. In this case, the classifier is able to classify most of the training examples correctly.

The expected training accuracy is generally very high, usually between **95% and 100%**. In many cases, the Decision Tree may even achieve nearly **100% training accuracy** because it learns the patterns and rules present in the training data very closely.

Why 100% Training Accuracy May Not Be Achieved

Even when the same dataset is used for both training and testing, 100% accuracy may not always be obtained due to the following reasons:

1. **Noisy Data**

Some records in the dataset may contain errors or unusual values.

2. **Conflicting Examples**

Two examples may have similar attribute values but different class labels (good/bad credit).

3. **Tree Restrictions**

Parameters such as maximum depth or minimum samples may limit the growth of the tree.

4. **Pruning**

Decision Trees are sometimes simplified to avoid overfitting, which may reduce training accuracy slightly.

5. **Data Overlap**

Some good and bad credit cases may share very similar attribute values, making perfect separation difficult.

5. **Solving the problem encountered in the previous question is using cross-validation? Describe what cross-validation is briefly. Train a Decision Tree again using cross-validation and report your results. Does your accuracy increase/decrease? Why?**

Program

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Load dataset
data = pd.read_csv("german_credit.csv")

# Convert categorical attributes to numeric
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Separate features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
```

```
# Create Decision Tree classifier
model = DecisionTreeClassifier(criterion='entropy')

# Perform 10-fold cross-validation
scores = cross_val_score(model, X, y, cv=10)

# Print results
print("Cross Validation Accuracy for each fold:")
print(scores)

print("\nAverage Accuracy:")
print(scores.mean() * 100)
```

Expected Output :

Cross Validation Accuracy for each fold:

[0.72 0.75 0.70 0.74 0.73 0.76 0.71 0.74 0.72 0.75]

Average Accuracy:

73.2

- 6. Another question might be, do you really need to input so many attributes to get good results? Maybe only a few would do. For example, you could try just having attributes 2, 3, 5, 7, 10, 17 (and 21, the class attribute (naturally)). Try out some combinations. (Remember to reload the arff data file to get all the attributes initially before you start selecting the ones you want.)**

Attribute Selection in Decision Tree Classification

In many data mining problems, all attributes may not be necessary for obtaining good classification results. Some attributes may contribute very little to the prediction process, while a few important attributes may be sufficient to build an accurate model. To study this, different combinations of attributes can be selected and the performance of the Decision Tree classifier can be compared.

For example, the following attributes may be selected from the German Credit Dataset:

Attribute Number	Attribute Name
2	credit_history
3	purpose
5	employment
7	debtors
10	housing
17	residence

The class attribute is:

- 21 → credit_risk (good/bad)

Program

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Load dataset
data = pd.read_csv("german_credit.csv")

# Convert categorical attributes into numeric values
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Select only required attributes
selected_columns = [
    'credit_history',
    'purpose',
    'employment',
    'debtors',
    'housing',
```



```

    'residence'
]

X = data[selected_columns]

# Target class attribute
y = data['class']

# Create Decision Tree classifier
model = DecisionTreeClassifier(criterion='entropy')

# Perform 10-fold cross validation
scores = cross_val_score(model, X, y, cv=10)

# Print results
print("Accuracy for each fold:")
print(scores)

print("\nAverage Accuracy:")
print(scores.mean() * 100)

```

Expected Output

Accuracy for each fold:

[0.69 0.71 0.72 0.68 0.70 0.73 0.69 0.71 0.70 0.72]

Average Accuracy:

70.5

7. **Train your Decision Tree again and report the Decision Tree and cross-validation results. Are they significantly different from results?**

Expected Output:

Decision Tree Using Selected Attributes

The Decision Tree classifier was trained again using only the selected attributes:

- credit_history
- purpose
- employment
- debtors
- housing
- residence

instead of using all attributes from the German Credit Dataset.

The model was evaluated using 10-fold cross-validation.

Cross-Validation Results

Using All Attributes

Measure	Result
Average Accuracy	Approximately 73% – 78%

Using Selected Attributes Only

Measure	Result
Average Accuracy	Approximately 68% – 72%

8. **Do you think it is a good idea to prefer simple decision trees instead of having long complex decision trees? How does the complexity of a Decision Tree relate to the bias of the model?**

Expected Output :

Simplicity vs Complexity in Decision Trees

In most cases it is better to prefer a simple Decision Tree rather than a very large and complex Decision Tree.

A simple Decision Tree:

- is easier to understand and interpret,
- requires less computation,
- generalizes better to new data,

- and reduces the chance of overfitting.

On the other hand, a very complex Decision Tree may memorize the training data instead of learning general patterns. This can reduce performance on unseen test data.

Relation Between Decision Tree Complexity and Bias

The complexity of a Decision Tree is closely related to the bias of the model.

1. Simple Decision Tree

A small or shallow tree has:

- fewer branches,
- fewer splitting conditions,
- and simpler rules.

Such a model usually has:

- high bias
- and low variance.

Because the model is too simple, it may fail to capture all patterns in the data. This problem is called:

Underfitting

2. Complex Decision Tree

A deep and large tree has:

- many branches,
- many detailed rules,
- and highly specific conditions.

Such a model usually has:

- low bias
- and high variance.

It fits the training data very closely and may memorize noise and outliers. This problem is called: Overfitting

Best Approach

The best model is usually a balanced Decision Tree that:

- is not too simple,
- and not too complex.

Techniques such as:

- pruning,
- limiting tree depth,
- and cross-validation

are used to obtain an optimal tree.

Simple Decision Trees are generally preferred because they are easier to interpret and generalize better to unseen data. However, overly simple trees may suffer from high bias and underfitting, while very complex trees may suffer from low bias and overfitting. Therefore, a balance between simplicity and complexity is important for building an effective Decision Tree model.

- 8. How make Decision Trees simpler by pruning the nodes. One approach is to use Reduced Error Pruning - Explain this idea briefly. Try reduced error pruning for training your Decision Trees using cross-validation and report the Decision Tree you obtain ? Also, report your accuracy using the pruned model. Does your accuracy increase?**

Decision Trees sometimes become very large and complex because they try to classify all training examples correctly. Such trees may overfit the training data and perform poorly on unseen data.

Pruning is a technique used to simplify the Decision Tree by removing unnecessary branches or nodes.

Reduced Error Pruning is a pruning method in which:

1. The complete Decision Tree is first constructed.
2. Nodes are removed one by one.
3. After removing a node, the accuracy is checked on validation data.
4. If removing the node does not reduce accuracy, the node is permanently removed.

The process continues until further pruning decreases the validation accuracy.

This method:

- reduces overfitting,
- produces simpler trees,
- improves interpretability,
- and may improve testing accuracy.

Program :

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
```

```

# Load dataset
data = pd.read_csv("german_credit.csv")

# Convert categorical attributes into numeric values
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Separate features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Create pruned Decision Tree
# max_depth acts as pruning control
model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=4,
    min_samples_leaf=10
)

# Perform 10-fold cross validation
scores = cross_val_score(model, X, y, cv=10)

# Train model
model.fit(X, y)

# Print accuracy
print("Cross Validation Accuracy:")
print(scores)

print("\nAverage Accuracy:")
print(scores.mean() * 100)

```

Expected Output:

Model	Average Accuracy
Unpruned Decision Tree	72% – 74%
Pruned Decision Tree	74% – 77%

10. How can you convert a Decision Trees into "if-then-else rules". Make up your own small Decision Tree consisting of 2-3 levels and convert it into a set of rules. There also exist different classifiers that output the model in the form of rules - one such classifier in Weka is rules.PART, train this model and report the set of rules obtained. Sometimes just one attribute can be good enough in making the decision, yes, just one ! Can you predict what attribute that might be in this dataset? OneR classifier uses a single attribute to make decisions (it chooses the attribute based on minimum error). Report the rule obtained by training a one R classifier. Rank the performance of j48, PART and oneR.

1. J48 Decision Tree Classifier

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import cross_val_score

# Load dataset
data = pd.read_csv("german_credit.csv")

# Encode categorical attributes
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# J48 equivalent Decision Tree
```

```

model = DecisionTreeClassifier(
    criterion='entropy'
)

# Cross-validation
scores = cross_val_score(model, X, y, cv=10)

# Train model
model.fit(X, y)

# Print Decision Tree
tree_rules = export_text(model, feature_names=list(X.columns))

print("J48 Decision Tree:\n")
print(tree_rules)

print("\nCross Validation Accuracy:")
print(scores)

print("\nAverage Accuracy:")
print(scores.mean() * 100)

```

2. PART Rule Classifier (Approximation)

Python sklearn does not directly provide PART classifier like Weka. We can approximate rule extraction from a Decision Tree.

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import cross_val_score

# Load dataset
data = pd.read_csv("german_credit.csv")

# Encode categorical attributes
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

```

```

# Features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Train shallow tree to generate readable rules
model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=3
)

# Cross-validation
scores = cross_val_score(model, X, y, cv=10)

# Train model
model.fit(X, y)

# Extract rules
rules = export_text(model, feature_names=list(X.columns))

print("PART-like Rule Set:\n")
print(rules)

print("\nAverage Accuracy:")
print(scores.mean() * 100)

```

3. OneR Classifier

OneR selects only one best attribute.

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import cross_val_score

# Load dataset
data = pd.read_csv("german_credit.csv")

# Encode categorical attributes
label_encoder = LabelEncoder()

```



```

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Select only one important attribute
X = data[['credit_history']]

# Target class
y = data.iloc[:, -1]

# Decision stump = OneR style classifier
model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=1
)

# Cross-validation
scores = cross_val_score(model, X, y, cv=10)

# Train model
model.fit(X, y)

# Print OneR rule
rules = export_text(model, feature_names=['credit_history'])

print("OneR Rule:\n")
print(rules)

print("\nAverage Accuracy:")
print(scores.mean() * 100)

```

Expected Output

Classifier	Expected Accuracy
J48	72% – 78%
PART	68% – 75%

Classifier	Expected Accuracy
-------------------	--------------------------

OneR	60% – 70%
------	-----------

11. Implement K-means clustering and Justify your answers.

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import (
    silhouette_score,
    confusion_matrix,
    accuracy_score
)

# Load dataset
data = pd.read_csv("german_credit.csv")

# Encode categorical attributes
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Features and target class
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply K-Means clustering
kmeans = KMeans(
    n_clusters=2,
    random_state=42,
```

```

        n_init=10
    )

    kmeans.fit(X_scaled)

    # Predicted cluster labels
    labels = kmeans.labels_

    # Cluster centroids
    centroids = kmeans.cluster_centers_

    # Since cluster labels may be reversed,
    # adjust labels for best matching

    accuracy1 = accuracy_score(y, labels)

    # Reverse cluster labels
    reversed_labels = 1 - labels

    accuracy2 = accuracy_score(y, reversed_labels)

    # Select maximum accuracy
    final_accuracy = max(accuracy1, accuracy2)

    # Print results
    print("Cluster Labels:\n")
    print(labels)

    print("\nCluster Centroids:\n")
    print(centroids)

    # Silhouette Score
    sil_score = silhouette_score(X_scaled, labels)

    print("\nSilhouette Score:")
    print(sil_score)

    # Confusion Matrix
    print("\nConfusion Matrix:")
    print(confusion_matrix(y, labels))

```

```
# Accuracy
print("\nAccuracy:")
print(final_accuracy * 100)

# Plot clusters
plt.figure(figsize=(8,6))

plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=labels
)

# Plot centroids
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker='X',
    s=200
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering on German Credit Dataset")

plt.show()
Expected Ouput:
Silhouette Score:

0.21

Confusion Matrix:

[[420 280]
 [160 140]]

Accuracy:

56.0
```

12. Implement SVM algorithm and report accuracy and compare the result with decision tree results.

Program :

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load dataset
data = pd.read_csv("german_credit.csv")

# Encode categorical attributes
label_encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == object:
        data[column] = label_encoder.fit_transform(data[column])

# Features and target class
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.3,
    random_state=42
)
```

```

# -----
# SVM Classifier
# -----
svm_model = SVC(
    kernel='rbf'
)

# Train SVM
svm_model.fit(X_train, y_train)

# Predict using SVM
svm_predictions = svm_model.predict(X_test)

# SVM Accuracy
svm_accuracy = accuracy_score(y_test, svm_predictions)

print("SVM Accuracy:")
print(svm_accuracy * 100)

print("\nSVM Confusion Matrix:")
print(confusion_matrix(y_test, svm_predictions))

print("\nSVM Classification Report:")
print(classification_report(y_test, svm_predictions))

# -----
# Decision Tree Classifier
# -----
dt_model = DecisionTreeClassifier(
    criterion='entropy'
)

# Train Decision Tree
dt_model.fit(X_train, y_train)

# Predict using Decision Tree
dt_predictions = dt_model.predict(X_test)

# Decision Tree Accuracy

```

```
dt_accuracy = accuracy_score(y_test, dt_predictions)
```

```
print("\nDecision Tree Accuracy:")
```

```
print(dt_accuracy * 100)
```

```
print("\nDecision Tree Confusion Matrix:")
```

```
print(confusion_matrix(y_test, dt_predictions))
```

```
print("\nDecision Tree Classification Report:")
```

```
print(classification_report(y_test, dt_predictions))
```

Expected Output :

SVM Accuracy:

77.0

SVM Confusion Matrix:

```
[[180 25]
```

```
[ 44 51]]
```

Decision Tree Accuracy:

72.0

Decision Tree Confusion Matrix:

```
[[165 40]
```

```
[ 44 51]]
```